

Python E-Course

Module 2 — Variables & Data Types

Detailed Notes

Module Overview

Level: Beginner

Duration: ~1 week (6-8 hours)

Prerequisites: Module 1

What You'll Learn

- Create and reassign variables that hold different kinds of data.
- Use Python's core built-in types: int, float, str, bool.
- Convert between types deliberately.
- Apply arithmetic, comparison, and logical operators.
- Read input from the user with `input()`.
- Produce well-formatted output with f-strings.

Lessons

#	Lesson	Duration
1	Variables and Assignment	25 min
2	Numbers: int and float	30 min
3	Strings and Booleans	30 min
4	Type Conversion	25 min
5	Operators	35 min
6	Input, Output & f-strings	30 min

Assessment

- Exercises (18) - Quiz (10 MCQ) - Assignment - Mini Project

What's Next

Module 3: Control Flow — let your programs make decisions and repeat work.

Lesson 1: Variables and Assignment

Module: 2 — Variables & Data Types

Duration: 25 minutes

Difficulty: Beginner

Learning Objectives

By the end of this lesson, you will be able to:

- Create variables and assign values to them.
- Reassign and update existing variables.
- Use multiple assignment in one statement.
- Inspect a variable's value and type.

Prerequisites

- Module 1 — All lessons

1. Why This Matters

A program without variables can only do one thing with one set of inputs. Variables let your code remember values, name them meaningfully, and operate on them. They are the most-used construct in every Python program you'll ever write.

2. Concept

2.1 What a variable is

A variable is a name that refers to a value stored in memory. You create one with =:

```
age = 25
```

This says: "the name age now refers to the value 25." Reading left-to-right (age = 25) is misleading — better to read it right-to-left: "compute 25, label it age."

2.2 Naming rules (refresher from Module 1)

- Use snake_case: user_age, not userAge.
- Start with a letter or _, not a digit.
- Case-sensitive: age and Age are different.
- Don't shadow built-ins: avoid list, dict, str, type, id as variable names.

2.3 Reassignment

A variable can hold a new value at any time — even a different type:

```
x = 5
x = 10
x = "hello"
```

Python is dynamically typed: the variable doesn't have a fixed type; the value does.

2.4 Multiple assignment

You can assign several variables in one line:

```
a, b, c = 1, 2, 3
```

Or give several variables the same value:

```
x = y = z = 0
```

Use these sparingly — only when they read naturally.

2.5 Checking types and values

`type(x)` returns the type of the value bound to `x`. `print(x)` shows its value.

```
n = 42
print(n)
print(type(n))
```

3. Code Examples

Example 1: Create and print variables

```
name = "Maya"
age = 28
print(name)
print(age)
```

Expected Output:

```
Maya
28
```

Explanation: Each line creates a variable; `print` shows its value.

Example 2: Reassign and change type

```
x = 5
print(x, type(x))
x = "five"
print(x, type(x))
```

Expected Output:

```
5 <class 'int'>
five <class 'str'>
```

Explanation: Python allows the variable's type to change between assignments.

Example 3: Multiple assignment

```
a, b, c = 10, 20, 30
print(a + b + c)
```

```
x = y = z = 0
print(x, y, z)
```

Expected Output:

```
00
0 0 0
```

Explanation: Three values are unpacked into three names. The second form assigns the same value to all three.

Example 4: Swapping values

```
a = 1
b = 2
a, b = b, a
print(a, b)
```

Expected Output:

```
2 1
```

Explanation: Python evaluates the right side first as (2, 1), then assigns. No temp variable needed.

4. Common Mistakes

Mistake	What Happens	Fix
5 = x instead of x = 5	SyntaxError	Name goes on the left
Using = to compare	Wrong result; later SyntaxError	Use == for comparison
Reading uninitialized variable	NameError	Assign before using
Shadowing built-ins: list = [1, 2]	Breaks later code using list()	Pick a different name

5. Practice Tasks

- Create a variable city set to your city's name and print it.
- Create three variables a, b, c with values 7, 8, 9 in one line. Print their sum.
- Swap two variables first and second without using a third.

6. Key Takeaways

- = assigns: name on the left, value on the right.
- Python is dynamically typed; a variable can hold any type.
- type(x) shows what a variable currently holds.
- a, b = b, a swaps cleanly without a temp.

7. What's Next

In the next lesson, we'll go deep on numbers — integers and floats — and the operations Python provides on them.

Lesson 2: Numbers — int and float

Module: 2 — Variables & Data Types

Duration: 30 minutes

Difficulty: Beginner

Learning Objectives

- Distinguish int from float and pick the right one.
- Use arithmetic operators including floor division and modulo.
- Use round() to control float precision in display.
- Recognize and handle floating-point inaccuracy.

Prerequisites

- Module 2 Lesson 1

1. Why This Matters

Numbers are everywhere: prices, scores, time, coordinates. Python has two main number types, and mixing them up — or trusting $0.1 + 0.2$ to equal 0.3 — causes subtle bugs in real systems.

2. Concept

2.1 int and float

- int — whole numbers, any size: 0, -7, 1000000000000000000.
- float — numbers with a decimal point: 3.14, -0.5, 1.0.

1 is an int; 1.0 is a float. They look similar, behave differently in division.

```
print(type(7))    # <class 'int'>
print(type(7.0))  # <class 'float'>
```

2.2 Arithmetic operators

Operator	Meaning	Example	Result
+	Add	$3 + 2$	5
-	Subtract	$3 - 2$	1
*	Multiply	$3 * 2$	6
/	True division	$7 / 2$	3.5
//	Floor division	$7 // 2$	3
%	Modulo (remainder)	$7 \% 2$	1
**	Exponent	$2 ** 3$	8

/ always returns a float, even when the result is whole: $4 / 2$ is 2.0, not 2.

2.3 Operator precedence

Same as math: $*$ > $,$ $/$, $//$, $\%$ > $+$, $-$. Use parentheses when in doubt:

```
print(2 + 3 * 4)      # 14, not 20
print((2 + 3) * 4)    # 20
```

2.4 Float precision

Floats use binary representation and cannot exactly represent some decimals:

```
print(0.1 + 0.2)      # 0.30000000000000004
```

Workarounds:

- For display, use `round(x, n)` — `round(0.1 + 0.2, 2) → 0.3`.
- For money, use the decimal module (covered in Module 10).

2.5 Useful built-ins

```
abs(-7)              # 7
round(3.567, 2)      # 3.57
pow(2, 8)             # 256 (same as 2 ** 8)
min(1, 5, 3)         # 1
max(1, 5, 3)         # 5
```

3. Code Examples

Example 1: Division flavors

```
print(10 / 3)
print(10 // 3)
print(10 % 3)
```

Expected Output:

```
3.3333333333333335
3
1
```

Explanation: `/` gives a float, `//` gives integer quotient, `%` gives remainder.

Example 2: Mixing int and float

```
print(3 + 1.0)
print(type(3 + 1.0))
```

Expected Output:

```
4.0
<class 'float'>
```

Explanation: When any operand is a float, the result promotes to float.

Example 3: Precision surprise

```
total = 0.1 + 0.2
print(total)
print(round(total, 2))
```

Expected Output:

```
0.30000000000000004
0.3
```

Explanation: Round for display; never compare floats with ==.

Example 4: Calculate seconds in a year

```
seconds = 60 * 60 * 24 * 365
print("Seconds per (non-leap) year:", seconds)
```

Expected Output:

```
Seconds per (non-leap) year: 31536000
```

Explanation: Pure int arithmetic; no precision loss.

4. Common Mistakes

Mistake	What Happens	Fix
Comparing floats with ==	Surprising False results	Use <code>abs(a - b) < 1e-9</code> or <code>math.isclose</code>
Using / when you meant //	Get a float when expecting int	Pick // for integer division
Forgetting precedence	<code>2 + 3 * 4</code> is 14, not 20	Use parentheses
Dividing by zero	<code>ZeroDivisionError</code>	Guard with if denominator != 0: (Module 3)

5. Practice Tasks

- Convert 100 minutes to hours-and-minutes: use // and %. Print like 1 hour 40 minutes.
- Compute the area of a circle of radius 5 (use 3.14159 for pi). Round to 2 decimals.
- Print `2 ** 16` and `2 ** 32` to see how fast powers grow.

6. Key Takeaways

- int for whole numbers, float for decimals.
- / is true division, // is floor division, % is remainder.
- Floats have precision quirks — round for display.
- Use parentheses when mixing operators.

7. What's Next

Numbers are only one half of "data." Next, we'll meet strings and booleans — text and true/false values.

Lesson 3: Strings and Booleans

Module: 2 — Variables & Data Types

Duration: 30 minutes

Difficulty: Beginner

Learning Objectives

- Create strings using single, double, and triple quotes.
- Concatenate and repeat strings.
- Use escape sequences like `\n` and `\t`.
- Create and compare boolean values.
- Recognize truthy and falsy values.

Prerequisites

- Module 2 Lesson 2

1. Why This Matters

Text is the data type you'll see most after numbers — names, messages, file paths, error text. Booleans (True / False) drive every decision your code makes. These two types appear in every program from this point on.

2. Concept

2.1 Creating strings

A string is a sequence of characters. Wrap them in matching quotes:

```
single = 'hello'
double = "hello"
triple = """hello
across
lines"""
```

Single and double quotes are equivalent — pick whichever avoids escaping. Triple quotes preserve newlines, great for multi-line text.

2.2 Escape sequences

Backslash starts a special character:

Sequence	Meaning
<code>\n</code>	Newline
<code>\t</code>	Tab
<code>\\</code>	Literal backslash
<code>\"</code>	Literal double quote
<code>\'</code>	Literal single quote

```
print("Line 1\nLine 2")
```

prints two lines.

2.3 Concatenation and repetition

```
greeting = "Hello, " + "World!"    # concatenation
line = "-" * 20                     # repetition: 20 dashes
```

You can't mix types with +. "age: " + 25 → TypeError. Convert: "age: " + str(25) (next lesson).

2.4 Raw strings

A leading r disables escapes — useful for Windows paths and regex:

```
path = r"C:\Users\Maya\Documents"
```

Without r, \U would be interpreted specially.

2.5 Booleans

Two values: True and False (capitalized — Python is case-sensitive).

Created by comparisons:

```
print(5 > 3)    # True
print(5 == 3)   # False
```

Or by direct assignment:

```
is_admin = True
```

2.6 Truthy and falsy

Every value behaves as True or False in a boolean context.

Falsy values: False, 0, 0.0, "" (empty string), None, [], {}, set().

Truthy values: everything else, including non-empty strings, non-zero numbers, and non-empty containers.

We'll use this constantly in if statements (Module 3).

3. Code Examples

Example 1: Quotes and escapes

```
print("It's a \"good\" day.")
print('She said "hi" to me.')
print("Path: C:\\Users\\Maya")
```

Expected Output:

```
It's a "good" day.
She said "hi" to me.
Path: C:\Users\Maya
```

Explanation: Pick the outer quote to avoid escaping the inner ones; escape backslashes with \\.

Example 2: Concatenation and repetition

```
name = "Sam"
greeting = "Hello, " + name + "!"
divider = "=" * 30
print(divider)
print(greeting)
print(divider)
```

Expected Output:

```
=====
Hello, Sam!
=====
```

Explanation: + joins strings; * repeats them N times.

Example 3: Triple-quoted string

```
poem = """Roses are red,
Violets are blue,
Python is fun,
And so are you."""
print(poem)
```

Expected Output:

```
Roses are red,
Violets are blue,
Python is fun,
And so are you.
```

Explanation: Triple quotes preserve newlines literally — no \n needed.

Example 4: Booleans from comparisons

```
age = 20
is_adult = age >= 18
print(is_adult)
print(type(is_adult))
```

Expected Output:

```
True
<class 'bool'>
```

Explanation: Comparison expressions produce bool values.

Example 5: Truthy and falsy with bool()

```
print(bool(0))
print(bool(42))
print(bool(""))
print(bool("hello"))
print(bool([]))
```

Expected Output:

```
False
True
False
True
False
```

Explanation: `bool(x)` shows whether `x` is truthy. Empty things → falsy.

4. Common Mistakes

Mistake	What Happens	Fix
Mismatched quotes: <code>"hi'</code>	<code>SyntaxError</code>	Match quotes
<code>"3" + 4</code>	<code>TypeError: can only concatenate str (not "int") to str</code>	<code>"3" + str(4)</code> (next lesson)
<code>true / false</code> lowercase	<code>NameError</code>	<code>True / False</code>
Forgetting <code>\\</code> in Windows paths	Path treated as escape sequence	Use <code>r"..."</code> raw strings

5. Practice Tasks

- Build a divider line of 40 underscores and print it above and below your name.
- Create a multi-line address using triple quotes and print it.
- Print whether the boolean expression `(5 > 3)` and `(10 < 20)` is `True` or `False`.

6. Key Takeaways

- Strings: single, double, or triple quotes — all valid.
- `+` concatenates strings; `*` repeats them.
- Use `\\` or raw strings for Windows paths.
- Booleans are `True/False` (capitalized).
- Most values are "truthy"; only `0`, `""`, `None`, empty containers, and `False` are "falsy".

7. What's Next

We've seen four core types — `int`, `float`, `str`, `bool`. Next: converting between them deliberately.

Lesson 4: Type Conversion

Module: 2 — Variables & Data Types

Duration: 25 minutes

Difficulty: Beginner

Learning Objectives

- Convert between int, float, str, and bool explicitly.
- Distinguish implicit (automatic) from explicit conversion.
- Anticipate ValueError when conversion fails.

Prerequisites

- Module 2 Lesson 3

1. Why This Matters

User input is always a string. Numeric data from files is often a string. You'll constantly need to convert. Doing it explicitly — and knowing what fails — is what separates fragile scripts from reliable ones.

2. Concept

2.1 Implicit conversion

Python automatically promotes ints to floats in mixed arithmetic:

```
result = 3 + 1.5    # int + float -> float
```

That's the only safe automatic conversion. Mixing str with anything else needs explicit conversion.

2.2 Explicit conversion

Function	Converts to	Example
int(x)	integer	int("42") → 42
float(x)	float	float("3.14") → 3.14
str(x)	string	str(7) → "7"
bool(x)	boolean	bool(0) → False

2.3 What can fail

- int("12.5") → ValueError (can't parse a decimal as int)
- int("hello") → ValueError
- float("hello") → ValueError
- str(x) and bool(x) never fail for basic types.

To go from float string to int, go through float:

```
int(float("12.5"))    # 12
```

2.4 Why explicit matters

```
age = "25"  
print(age + 5)          # TypeError  
print(int(age) + 5)     # 30
```

Make conversion intentional. Don't rely on Python to "figure it out" — Python won't.

3. Code Examples

Example 1: String to int and back

```
age_str = "30"  
age = int(age_str)  
print(age + 5)  
back = str(age)  
print("Age is " + back)
```

Expected Output:

```
35  
Age is 30
```

Explanation: `int()` parses the string; `str()` converts back for concatenation.

Example 2: Float to int truncates

```
print(int(3.9))  
print(int(-2.7))
```

Expected Output:

```
3  
-2
```

Explanation: `int()` chops the decimal — it does not round. Use `round()` if you want rounding.

Example 3: Booleans are numbers

```
print(int(True))  
print(int(False))  
print(True + True)
```

Expected Output:

```
1  
0  
2
```

Explanation: `True` is 1, `False` is 0 under the hood. Rarely used directly but good to know.

Example 4: When conversion fails

```
print(int("abc"))
```

Expected Output:

```
ValueError: invalid literal for int() with base 10: 'abc'
```

Explanation: Strings must look like a number to convert. Handle this with try/except (Module 8).

4. Common Mistakes

Mistake	What Happens	Fix
Adding string + number	TypeError	Convert one side explicitly
<code>int("3.5")</code>	ValueError	<code>int(float("3.5"))</code>
Trusting <code>int()</code> to round	<code>int(3.9)</code> is 3, not 4	Use <code>round(3.9)</code>
Forgetting input is always a string	Math fails on user input	Wrap <code>input()</code> with <code>int()</code> or <code>float()</code>

5. Practice Tasks

- Convert "100" to int, multiply by 2, then convert back to a string and print.
- Convert 3.7 to int and predict the output. Verify.
- Write `bool(x)` for: 0, "0", "", "False". Predict before running.

6. Key Takeaways

- Conversions: `int()`, `float()`, `str()`, `bool()`.
- Python won't convert silently between strings and numbers.
- `int()` truncates floats; use `round()` to round.
- `int(string)` fails if the string isn't a valid integer.

7. What's Next

We've covered values. Next, operators — how to combine values with arithmetic, comparison, and logic.

Lesson 5: Operators

Module: 2 — Variables & Data Types

Duration: 35 minutes

Difficulty: Beginner

Learning Objectives

- Use arithmetic, comparison, logical, assignment, and identity operators.
- Combine them into expressions with the correct precedence.
- Distinguish == from is.

Prerequisites

- Module 2 Lesson 4

1. Why This Matters

Operators are how data combines. Every conditional, every computation, every decision your program makes flows through them. Get the categories straight now and the rest of Python clicks.

2. Concept

2.1 Arithmetic operators (recap)

+, -, , /, //, %, * — covered in Lesson 2.

2.2 Comparison operators

Return True or False.

Operator	Meaning
==	Equal
!=	Not equal
<, <=	Less than / less-or-equal
>, >=	Greater / greater-or-equal

```
print(5 == 5)    # True
print(5 != 3)    # True
```

2.3 Logical operators

Combine boolean expressions.

Operator	Result is True if...
and	both operands are true
or	at least one is true
not	the operand is false

```
age = 20
has_id = True
can_enter = age >= 18 and has_id
```

and and or are short-circuit: Python stops evaluating once the result is decided.

2.4 Assignment operators

Beyond =, Python offers shortcuts:

Operator	Equivalent
x += 1	x = x + 1
x -= 1	x = x - 1
x *= 2	x = x * 2
x /= 2	x = x / 2
x //= 2	x = x // 2
x %= 3	x = x % 3
x **= 2	x = x ** 2

2.5 Identity operators

- is — checks if two names refer to the same object in memory.
- is not — opposite.

```
x = None
if x is None:
    print("nothing")
```

Use == for value equality, is only for None, True, False, and identity checks. 5 == 5 is correct; 5 is 5 happens to be True in CPython but isn't guaranteed.

2.6 Membership operators (preview)

- in, not in — used heavily with strings and collections (Modules 5-6).

```
print("py" in "python")    # True
```

2.7 Precedence (high to low, simplified)

- **
- unary -, not
- *, /, //, %
- +, -
- <, <=, >, >=, ==, !=
- and
- or

When unsure, add parentheses. Readability beats cleverness.

3. Code Examples

Example 1: Logical combination

```
age = 22
has_ticket = True
print(age >= 18 and has_ticket)
```

```
print(age < 18 or has_ticket)
print(not has_ticket)
```

Expected Output:

```
True
True
False
```

Explanation: Both and and or evaluate left to right and short-circuit.

Example 2: Compound assignment

```
score = 0
score += 10
score += 5
score *= 2
print(score)
```

Expected Output:

```
30
```

Explanation: += and = accumulate in-place; final value is (0 + 10 + 5) * 2.

Example 3: == vs is

```
a = [1, 2]
b = [1, 2]
print(a == b)
print(a is b)
```

Expected Output:

```
True
False
```

Explanation: Same contents (==) but different objects (is). Two separate lists were created.

Example 4: Precedence trap

```
print(2 + 3 * 4)
print((2 + 3) * 4)
print(not 5 > 3)
print(not (5 > 3))
```

Expected Output:

```
14
20
False
False
```

Explanation: not has higher precedence than >, but here both not 5 (truthy) and not (5 > 3) (not True) end up False. Parens make intent obvious.

4. Common Mistakes

Mistake	What Happens	Fix
= instead of ==	SyntaxError in conditions	Use == to compare
Using is for value equality	Sometimes True, sometimes False — unreliable	Use == for values, is for None
Chained comparisons misread	$1 < x < 10$ is valid; $1 < x$ and $x < 10$ is the long form	Both work; prefer the chained form
Operator precedence assumptions	Surprising results	Add parentheses

5. Practice Tasks

- Predict and verify: True and False or True.
- Use += to accumulate $1 + 2 + 3 + 4 + 5$ into a variable total. Print it.
- Use chained comparison to check whether a variable x is between 1 and 100 (inclusive).

6. Key Takeaways

- Categories: arithmetic, comparison, logical, assignment, identity, membership.
- and/or short-circuit.
- == for values, is for object identity (mostly None).
- Parens beat memorizing precedence.

7. What's Next

Final lesson of this module: how to take input from the user and produce nicely-formatted output with f-strings.

Lesson 6: Input, Output & f-strings

Module: 2 — Variables & Data Types

Duration: 30 minutes

Difficulty: Beginner

Learning Objectives

- Read user input from the terminal with `input()`.
- Convert input to the right type before using it.
- Format output cleanly with f-strings, including width and precision.
- Choose between f-strings, `.format()`, and `%-formatting`.

Prerequisites

- Module 2 Lesson 5

1. Why This Matters

Until now, your programs only print pre-set values. Real programs talk to users — ask for input, transform it, and show results. f-strings are how modern Python builds readable output, and you'll use them constantly.

2. Concept

2.1 The `input()` function

`input(prompt)` displays the prompt and waits for the user to type a line. It returns a string — always.

```
name = input("What is your name? ")
print("Hello,", name)
```

If you need a number:

```
age = int(input("Age: "))
```

2.2 f-strings (formatted string literals)

Prefix a string with `f` (or `F`). Inside `{}` you can put any Python expression:

```
name = "Sam"
age = 30
print(f"Hi {name}, you are {age} years old.")
```

f-strings (Python 3.6+) are the recommended way to format strings. They're readable and fast.

2.3 Format specifiers

After a `:` inside the braces, you can control format:

Spec	Meaning	Example	Output
------	---------	---------	--------

:.2f	Float with 2 decimals	f"{3.14159:.2f}"	3.14
:>10	Right-align in width 10	f"{'hi':>10}"	' hi'
:<10	Left-align in width 10	f"{'hi':<10}"	'hi '
:^10	Center in width 10	f"{'hi':^10}"	' hi '
:,	Thousands separator	f"{1000000:,}"	'1,000,000'
:.0%	Percentage	f"{0.85:.0%}"	'85%'
:08d	Pad int with zeros	f"{42:08d}"	'00000042'

2.4 Older formatting styles

You'll see these in old code:

```
"Hi %s, you are %d" % ("Sam", 30)      # %-formatting
"Hi {}, you are {}".format("Sam", 30)  # str.format
f"Hi {name}, you are {age}"            # f-string (modern, recommended)
```

For new code, use f-strings.

2.5 Print's keyword arguments (recap)

```
print("a", "b", "c", sep=", ", end="!\n")
```

- `sep` — separator between arguments (default space).
- `end` — what to print at the end (default newline).

3. Code Examples

Example 1: Read a name and greet

```
name = input("What's your name? ")
print(f"Hello, {name}!")
```

Sample interaction:

```
What's your name? Alex
Hello, Alex!
```

Explanation: Whatever the user types becomes the string `name`.

Example 2: Read numbers and compute

```
a = int(input("First number: "))
b = int(input("Second number: "))
print(f"{a} + {b} = {a + b}")
```

Sample interaction:

```
First number: 12
Second number: 8
12 + 8 = 20
```

Explanation: `input()` returns a string; `int()` converts it. The f-string embeds the math directly.

Example 3: Currency formatting

```
price = 1499.5
tax = 0.18
total = price * (1 + tax)
print(f"Subtotal: ₹{price:,.2f}")
print(f"Tax:      {tax:.0%}")
print(f"Total:    ₹{total:,.2f}")
```

Expected Output:

```
Subtotal: ₹1,499.50
Tax:      18%
Total:    ₹1,769.41
```

Explanation: `,.2f` adds thousand separators and 2 decimals; `.0%` shows percent.

Example 4: Aligned table

```
items = [("Apple", 50), ("Bread", 40), ("Milk", 60)]
print(f"{'Item':<10}{'Price':>6}")
print("-" * 16)
for item, price in items:
    print(f"{'item':<10}{'price':>6}")
```

Expected Output:

```
Item      Price
-----
Apple      50
Bread      40
Milk       60
```

Explanation: `<10` and `>6` align columns to fixed widths. (The for loop comes formally in Module 3 — included here for the layout.)

Example 5: Padding numbers

```
for n in [1, 12, 123]:
    print(f"ID-{n:04d}")
```

Expected Output:

```
ID-0001
ID-0012
ID-0123
```

Explanation: `04d` pads with leading zeros to width 4.

4. Common Mistakes

Mistake	What Happens	Fix
Forgetting the <code>f</code> prefix	Braces appear literally	Add <code>f</code> before the quote
<code>input()</code> returning a string when you need a number	Math fails	Wrap with <code>int()</code> or <code>float()</code>

Using .2f on a string	ValueError	Convert to float first
Mixing single and double quotes inside an f-string	Backticks-of-quotes confusion	Use the other quote inside {} than outside

5. Practice Tasks

- Ask the user for their first and last name and print Welcome, FIRST LAST! using an f-string.
- Ask for a number n and print its square, cube, and square root ($n^{0.5}$), each labeled.
- Print the numbers 1, 5, and 25 right-aligned in a column of width 6.

6. Key Takeaways

- `input()` returns a string; convert when needed.
- f-strings (f"...") embed expressions in {}.
- Format specifiers after : control width, alignment, decimals, padding, percent.
- Prefer f-strings over `.format()` and %-formatting in new code.

7. What's Next

Module 2 ends here. Up next: Module 3 — Control Flow. With variables, types, and operators in hand, you'll now make your programs decide and repeat.

Module 2 — Exercises

18 exercises (3 per lesson). Save each as a .py file.

Lesson 2.1 — Variables and Assignment

Exercise 1: Personal data (Easy)

Task: Create variables name, age, country. Print each on its own line.

Expected Output:

```
Maya
22
India
```

Exercise 2: Swap (Medium)

Task: Assign a = 100, b = 200. Swap their values without a temp variable and print both.

Expected Output:

```
a=200 b=100
```

Exercise 3: Multi-assign (Hard)

Task: Use one line to assign three variables x, y, z to the values 5, 10, 15. Then in a second line set all three to 0 using chained assignment. Print all six results in order.

Expected Output:

```
5 10 15
0 0 0
```

Lesson 2.2 — Numbers

Exercise 1: Quotient & remainder (Easy)

Task: Print the quotient and remainder when 47 is divided by 6.

Expected Output: 7 5

Exercise 2: Circle area (Medium)

Task: Use radius = 7 and pi = 3.14159. Print the area rounded to 2 decimal places.

Expected Output: 153.94

Exercise 3: Compound interest (Hard)

Task: Principal=10000, rate=0.08, years=5. Compute compound interest: $P (1 + r)^{\text{years}} - P$. Print rounded to 2 decimals.

Expected Output: 4693.28

Lesson 2.3 — Strings and Booleans

Exercise 1: Build a divider (Easy)

Task: Print 30 hyphens, your name centered between two more dividers.

Expected Output:

```
-----  
Maya  
-----
```

Exercise 2: Escape chars (Medium)

Task: Print exactly: She said "Python's great!" — using escapes correctly.

Expected Output: She said "Python's great!"

Exercise 3: Boolean drill (Hard)

Task: Print the boolean result of each: `bool(0)`, `bool("0")`, `bool(" ")`, `bool([])`, `bool("False")`. One per line.

Expected Output:

```
False  
True  
True  
False  
True
```

Lesson 2.4 — Type Conversion

Exercise 1: String to int (Easy)

Task: Convert "15" to int, add 10, print.

Expected Output: 25

Exercise 2: Float to int (Medium)

Task: Convert 9.99 to int and predict the result. Then convert -3.7 to int. Print both.

Expected Output:

```
9  
-3
```

Exercise 3: Round-trip (Hard)

Task: Start with "42.5". Convert to float, then to int, then to string, then concatenate " is the answer" and print.

Expected Output: 42 is the answer

Lesson 2.5 — Operators

Exercise 1: Compound (Easy)

Task: Start with `x = 5`. Use `*=`, `+=`, `-=` to get `x` to 19. Print after each step.

Expected Output (one possible path):

```
5
20
22
19
```

(Sample path: `x *= 4 → 20`; `x += 2 → 22`; `x -= 3 → 19`.)

Exercise 2: Logic table (Medium)

Task: Print the result of: `True and False`, `True or False`, `not True`, `(5 > 3)` and `(10 < 5)`. One per line.

Expected Output:

```
False
True
False
False
```

Exercise 3: `==` vs `is` (Hard)

Task: Compare two equal lists `[1, 2, 3]` with `==` and with `is`. Print both results.

Expected Output:

```
True
False
```

Lesson 2.6 — Input, Output & f-strings

Exercise 1: Greet (Easy)

Task: Prompt for a name. Print `Hello, NAME!` using an f-string.

Sample:

```
Name: Sam
Hello, Sam!
```

Exercise 2: Tip calculator (Medium)

Task: Prompt for `bill` and `tip_percent`. Print the tip and the total, both formatted to 2 decimals.

Sample:

```
Bill: 1200
Tip %: 15
Tip: 180.00
Total: 1380.00
```

Exercise 3: Aligned receipt (Hard)

Task: Hard-code three items with names and prices. Print them in two aligned columns (name left, price right, width 20 total).

Expected Output:

Apple	50
Whole Wheat Bread	60
Milk Carton	100

Module 2 — Quiz

10 multiple-choice questions covering Lessons 1-6.

Question 1

What does `type(3.0)` return?

A. `<class 'int'>` B. `<class 'float'>` C. `<class 'number'>` D. `<class 'double'>`

Answer: B

Explanation: Any number with a decimal is a float. (Lesson 2)

Question 2

What does `7 // 2` evaluate to?

A. 3.5 B. 3 C. 4 D. 3.0

Answer: B

Explanation: `//` is floor division — drops the decimal. (Lesson 2)

Question 3

Which is a falsy value?

A. `"False"` B. `1` C. `[0]` D. `""`

Answer: D

Explanation: Empty string is falsy. The string `"False"` is non-empty, so truthy. (Lesson 3)

Question 4

What does `int("3.7")` produce?

A. 3 B. 4 C. 3.7 D. `ValueError`

Answer: D

Explanation: `int()` can parse only integer strings. Use `int(float("3.7"))` to get 3. (Lesson 4)

Question 5

What is the value of `bool("")` ?

A. `True` B. `False` C. `None` D. `ValueError`

Answer: B

Explanation: Empty string is falsy. (Lesson 3)

Question 6

Which operator returns the remainder of division?

A. / B. // C. % D. **

Answer: C

Explanation: % is the modulo operator. (Lesson 2)

Question 7

What is the output of `print(5 == 5.0)` ?

A. True B. False C. TypeError D. None

Answer: A

Explanation: Python compares numeric values, not types. (Lesson 5)

Question 8

What does `input()` always return?

A. an int B. a float C. a string D. depends on what was typed

Answer: C

Explanation: `input()` always returns a str. Convert if you need a number. (Lesson 6)

Question 9

Which f-string format prints 0042?

A. `f"{42:4d}"` B. `f"{42:04d}"` C. `f"{42:.4f}"` D. `f"{42:>4}"`

Answer: B

Explanation: 04d pads with zeros to width 4. (Lesson 6)

Question 10

What does this print?

```
x = 10
x += 5
x *= 2
print(x)
```

A. 15 B. 25 C. 30 D. 40

Answer: C

Explanation: $10 + 5 = 15$, then $15 * 2 = 30$. (Lesson 5)

Module 2 — Assignment: Unit Converter

Estimated time: 1 hour

Combines concepts from: Module 2 + Module 1

Brief

Build a simple unit converter that prompts the user for a value and converts it across several units. The goal is to apply variables, type conversion, arithmetic operators, `input()`, and f-string formatting in one cohesive script.

Requirements

The program must:

- Greet the user and explain what it does (one or two lines using `print` or f-strings).
- Ask for a temperature in Celsius and print the equivalent in Fahrenheit and Kelvin, both rounded to 2 decimals.
- Ask for a distance in kilometers and print the equivalent in miles ($1 \text{ km} \approx 0.621371 \text{ mi}$), rounded to 2 decimals.
- Ask for a weight in kilograms and print the equivalent in pounds ($1 \text{ kg} \approx 2.20462 \text{ lb}$), rounded to 2 decimals.
- End with a polite thank-you line.

Constraints

- Use only Module 1-2 concepts.
- No if, no loops, no functions.
- Use f-strings for output.
- Convert each user input to float immediately after `input()`.

Expected Behavior

```
Welcome to the unit converter.
```

```
Temperature in Celsius: 25
25.00 °C = 77.00 °F = 298.15 K
```

```
Distance in kilometers: 10
10.00 km = 6.21 miles
```

```
Weight in kilograms: 70
70.00 kg = 154.32 lb
```

```
Thanks for using the converter!
```

Hints

- Fahrenheit: $f = c * 9/5 + 32$.
- Kelvin: $k = c + 273.15$.

- Use `f"{value:.2f}"` to format to 2 decimals.

Grading Rubric

Criterion	Weight
Correct math	35%
f-strings used	20%
Type conversion correct	20%
Output formatted neatly	15%
PEP 8 style	10%

Submission

Save as `assignment_module2.py`.

Module 2 — Mini Project: Tip & Bill Splitter

Overview

A small interactive program that takes a bill amount, tip percentage, and number of people, and prints a clean per-person breakdown. Practices `input()`, type conversion, arithmetic, and f-strings.

Concepts Used

- Variables and types (Module 2 L1, L2)
- `input()` and conversion (Module 2 L4, L6)
- Arithmetic operators (Module 2 L5)
- f-string formatting (Module 2 L6)

Features

- Ask for the total bill in rupees.
- Ask for tip percentage (e.g. 10, 15, 20).
- Ask for the number of people.
- Print a formatted receipt showing: subtotal, tip amount, total, per-person share.

Starter Code

Save as `tip_splitter.py`:

```
# Mini Project: Tip & Bill Splitter
# Author: [Your Name]

print("=" * 40)
print("      TIP & BILL SPLITTER")
print("=" * 40)

bill = float(input("Total bill (₹): "))
tip_percent = float(input("Tip %: "))
people = int(input("Number of people: "))

# TODO: compute tip, total, and per-person share, then print
```

Expected Interaction

```
=====
      TIP & BILL SPLITTER
=====
Total bill (₹): 2400
Tip %: 15
Number of people: 4

-----
Subtotal:      ₹2,400.00
Tip (15%):     ₹360.00
Total:         ₹2,760.00
Per person:    ₹690.00
-----
```

Extension Challenges

- Add a "service charge" prompt (a flat amount on top of the tip).
- Show the per-person tip and per-person subtotal as separate lines.
- Print a thank-you line including the user's name (ask at the start).

Grading Rubric

Criterion	Weight
Reads input correctly	25%
Math is correct	30%
Output is well-formatted	25%
PEP 8 + clear variable names	20%